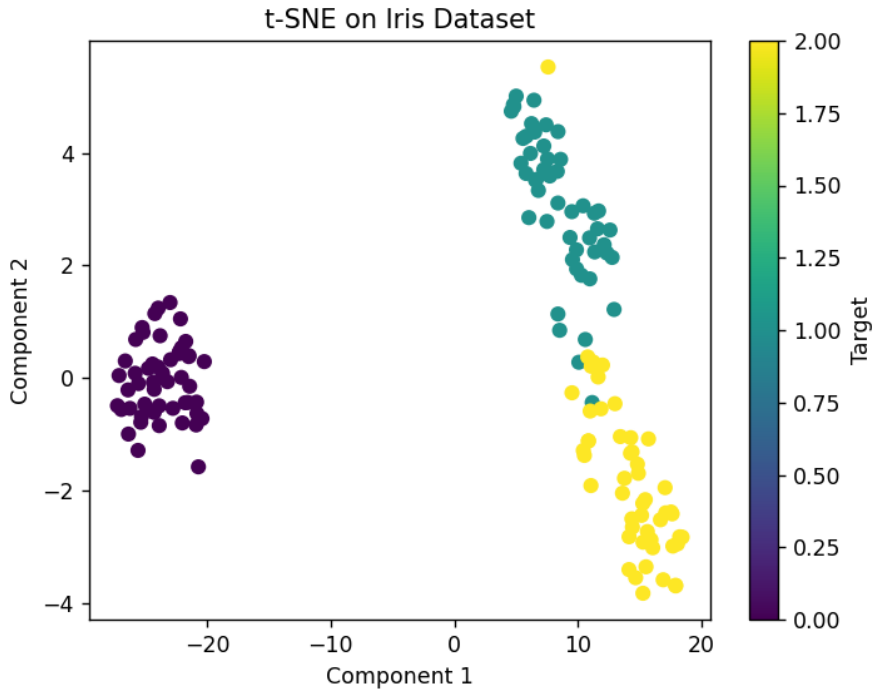
MINOR ASSIGNMENT-4: MACHINE LEARNING- CLASSIFICATION, REGRESSION AND CLUSTERING

1. Perform dimensionality reduction using scikit-learn's TSNE estimator on the Iris dataset, then graph the results.

```
#Name-Arman Singh Regd-2241019468
from sklearn.datasets import load_iris
from sklearn.manifold import TSNE
import matplotlib.pyplot as plt

iris = load_iris()
tsne = TSNE(n_components=2, random_state=42)
X_reduced = tsne.fit_transform(iris.data)

plt.scatter(X_reduced[:, 0], X_reduced[:, 1], c=iris.target, cmap='viridis')
plt.title("t-SNE on Iris Dataset")
plt.xlabel("Component 1")
plt.ylabel("Component 2")
plt.colorbar(label="Target")
plt.show()
```

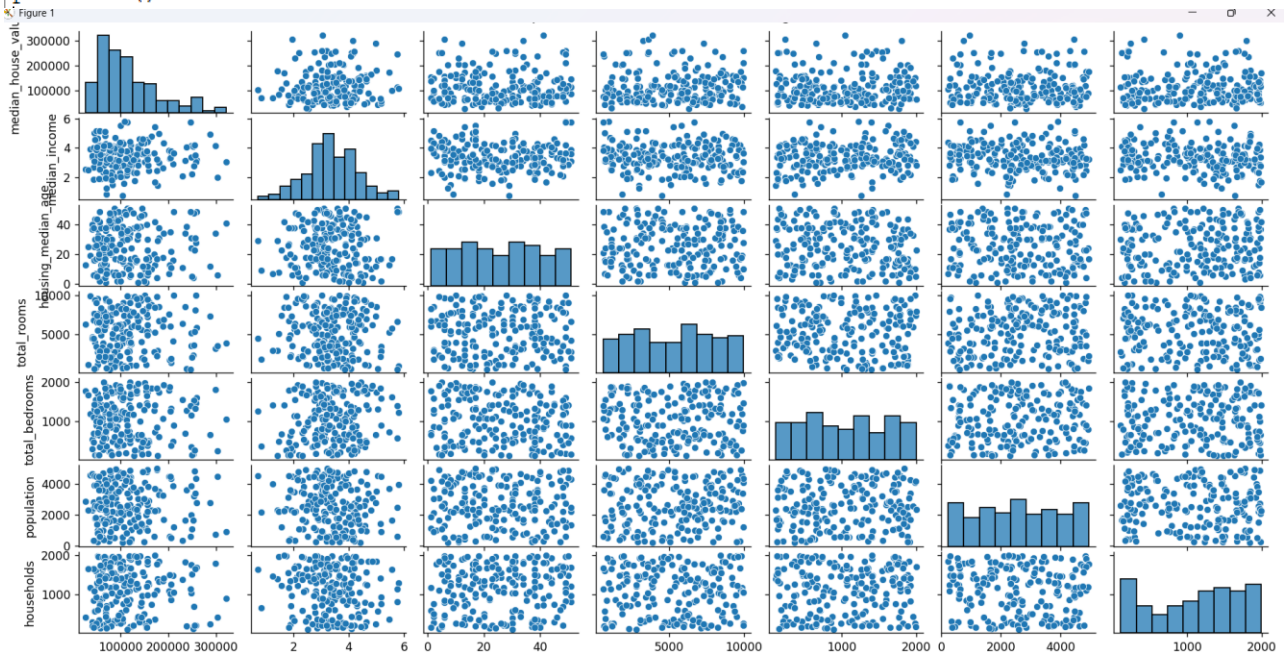
Output



2. Create a Seaborn pairplot graph for the California Housing dataset. Try the Matplotlib features to panning and zoom in on the diagram. These are accessible via the icons in the Matplotlib window.

Ans-

```
#Name-Arman Singh Regd-2241019468
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np

# Simulate a small dataset based on the given structure
np.random.seed(0)  # for reproducibility
n = 200  # simulate 200 records

# Simulating variables around expected means from the description
data = {
    'median_house_value': np.exp(np.random.normal(11.49, 0.5, n)),  # undo log s
    'median_income': np.random.normal(3.5, 1.0, n),  # median income (approx. lo
    'housing_median_age': np.random.randint(1, 52, n),  # 1-51 years
    'total_rooms': np.random.randint(500, 10000, n),
    'total_bedrooms': np.random.randint(100, 2000, n),
    'population': np.random.randint(200, 5000, n),
    'households': np.random.randint(100, 2000, n),
    'latitude': np.random.uniform(32, 42, n),  # California lat range
    'longitude': np.random.uniform(-124, -114, n)  # California long range
}

df = pd.DataFrame(data)

# Optional: show the first few rows
print("Sample of the simulated dataset:")
print(df.head())

# Create a Seaborn pairplot
sns.pairplot(df[['median_house_value', 'median_income', 'housing_median_age',
                 'total_rooms', 'total_bedrooms', 'population', 'households']])
plt.suptitle("Seaborn Pairplot for Simulated California Housing Dataset", y=1.02

# Show interactive plot with pan/zoom tools (requires interactive backend)
plt.show()
```

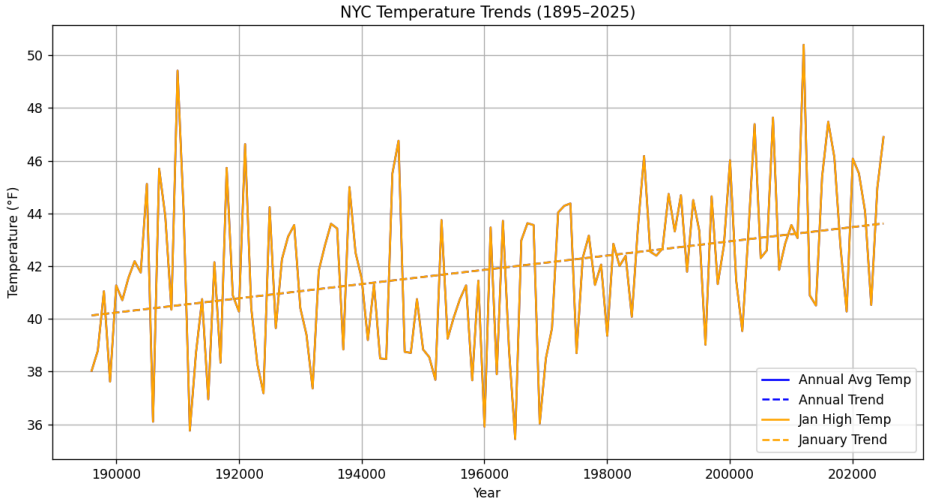


3.Go to NOAA's Climate at a Glance page (Link) and download the available time series data for the average annual temperatures of New York City from 1895 to today (1895-2025). Implement simple linear regression using average annual temperature data. Also, show how does the temperature trend compare to the average January high temperatures?

Ans-

```
#Name-Arman Singh Regd-2241019468
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
import numpy as np
# Load the datasets
annual = pd.read_csv("D:/BIter 3rd Year 6th Sem/Python/MyAssignment/Ass4/annual_avg_temp.
january = pd.read_csv("D:/BIter 3rd Year 6th Sem/Python/MyAssignment/Ass4/jan_avg_temp.cs
# Rename columns for clarity
annual.columns = ['Year', 'AnnualAvgTemp']
january.columns = ['Year', 'JanHighTemp']
# Merge both datasets on 'Year'
df = pd.merge(annual, january, on='Year')
df.dropna(inplace=True)
# Prepare data for regression
X = df['Year'].values.reshape(-1, 1)
y_annual = df['AnnualAvgTemp'].values
y_january = df['JanHighTemp'].values
# Linear regression for Annual Avg Temp
model_annual = LinearRegression()
model_annual.fit(X, y_annual)
pred_annual = model_annual.predict(X)
# Linear regression for Jan High Temp
model_january = LinearRegression()
model_january.fit(X, y_january)
pred_january = model_january.predict(X)
# Plotting
plt.figure(figsize=(12, 6))
plt.plot(df['Year'], y_annual, label='Annual Avg Temp', color='blue')
plt.plot(df['Year'], pred_annual, linestyle='--', color='blue', label='Annual Trend')
plt.plot(df['Year'], y_january, label='Jan High Temp', color='orange')
plt.plot(df['Year'], pred_january, linestyle='--', color='orange', label='January Trend')
plt.xlabel('Year')
plt.ylabel('Temperature (°F)')
plt.title('NYC Temperature Trends (1895-2025)')
plt.legend()
plt.grid(True)
plt.show()
# Trend Comparison
print("Annual Avg Temp Trend (slope):", model_annual.coef_[0])
print("January High Temp Trend (slope):", model_january.coef_[0])
```

Output:



4. Load the Iris dataset from the scikit-learn library and perform classification on it with the k-nearest neighbors algorithm. Use a KNeighborsClassifier with the default k value. What is the prediction accuracy?

Ans-

```
#Name-Arman Singh Regd-2241019468
from sklearn.datasets import load_iris
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Load Iris dataset
iris = load_iris()
X, y = iris.data, iris.target
# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
# KNN with default k=5
knn = KNeighborsClassifier()
knn.fit(X_train, y_train)
y_pred = knn.predict(X_test)
# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f"Prediction Accuracy: {accuracy:.4f}")
```

```
Iter 3rd Year 6th Sem\Python\MyAssignment\Ass4\q4.py"
Prediction Accuracy: 1.0000
```

5. You are given a dataset of 2D points with their corresponding class labels. The dataset is as follows:

Point ID	x	y	Class
A	2.0	3.0	0
B	1.0	1.0	0
C	4.0	4.0	1
D	5.0	2.0	1

A new point P with coordinates $(3.0, 3.0)$ needs to be classified using the KNN algorithm. Use the Euclidean distance to calculate the distance between points.

```
#Name -Arman Singh Regd=2241019468
import numpy as np
points = {
    'A': (2.0, 3.0, 0),
    'B': (1.0, 1.0, 0),
    'C': (4.0, 4.0, 1),
    'D': (5.0, 2.0, 1)
}
P = (3.0, 3.0)

distances = []
for label, (x, y, cls) in points.items():
    distance = np.sqrt((x - P[0])**2 + (y - P[1])**2)
    distances.append((label, distance, cls))

distances.sort(key=lambda x: x[1])
nearest_classes = [cls for _, _, cls in distances[:3]]
prediction = max(set(nearest_classes), key=nearest_classes.count)
print("Predicted Class of P:", prediction)
```

Output:

```
Iter 3rd Year 6th Sem\Python\MyAssignment\Ass4\q5.py"
Predicted Class of P: 1
```

6. A teacher wants to classify students as "Pass" or "Fail" based on their performance in three exams. The dataset includes three features:

Exam 1 Score	Exam 2 Score	Exam 3 Score	Class (Pass/Fail)
85	90	88	Pass
70	75	80	Pass
60	65	70	Fail
50	55	58	Fail
95	92	96	Pass
45	50	48	Fail

A new student has the following scores:

- Exam 1 Score: 72
- Exam 2 Score: 78
- Exam 3 Score: 75

Classify this student using the K-Nearest Neighbors (KNN) algorithm with $k = 3$.

```python
#Name -Arman Singh Regd=2241019468
from sklearn.neighbors import KNeighborsClassifier

X = [[85, 90, 88], [70, 75, 80], [60, 65, 70],
     [50, 55, 58], [95, 92, 96], [45, 50, 48]]
y = ['Pass', 'Pass', 'Fail', 'Fail', 'Pass', 'Fail']

knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X, y)

new_student = [[72, 78, 75]]
prediction = knn.predict(new_student)
print("Predicted Class for new student:", prediction[0])
```

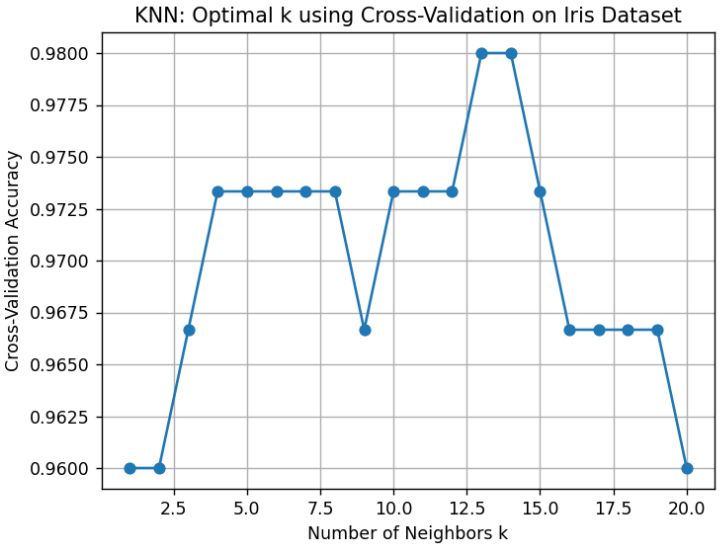
Output:

```
Iter 3rd Year 6th Sem\Python\MyAssignment\Ass4\q6.py"
Predicted Class for new student: Pass
```

7. Using scikit-learn's KFold class and the cross val score function, determine the optimal value for k to classify the Iris dataset using a KNeighborsClassifier

```python
#Name -Arman Singh Regd=2241019468
from sklearn.datasets import load_iris
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import KFold, cross_val_score
import matplotlib.pyplot as plt

# Load Iris dataset
iris = load_iris()
X = iris.data
y = iris.target

# Range of k values to try
k_values = range(1, 21)
cv_scores = []

# Use 5-fold cross-validation
kf = KFold(n_splits=5, shuffle=True, random_state=42)

# Evaluate accuracy for each k
for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k)
    scores = cross_val_score(knn, X, y, cv=kf)
    cv_scores.append(scores.mean())
    print(f"k = {k}, Cross-Validation Accuracy = {scores.mean():.4f}")

# Plotting k vs accuracy
plt.plot(k_values, cv_scores, marker='o')
plt.xlabel('Number of Neighbors k')
plt.ylabel('Cross-Validation Accuracy')
plt.title('KNN: Optimal k using Cross-Validation on Iris Dataset')
plt.grid(True)
plt.show()

# Best k
best_k = k_values[cv_scores.index(max(cv_scores))]
print(f"\n✅ Optimal k: {best_k} with Accuracy: {max(cv_scores):.4f}")
```

Output:

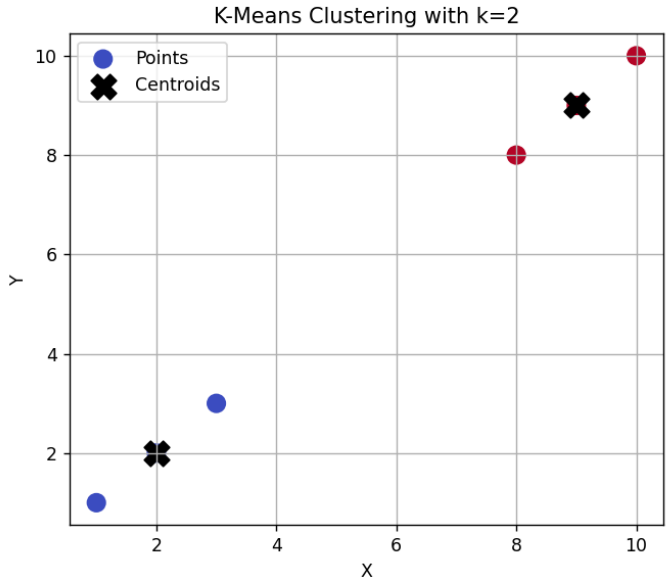


8. . Write a Python script to perform K-Means clustering on the following dataset: Dataset: {(1,1),(2,2),(3,3),(8,8),(9,9),(10,10)} Use k=2 and visualize the clusters.

Ans-

```
#Name -Arman Singh Regd=2241019468
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
# Dataset
X = np.array([[1, 1], [2, 2], [3, 3], [8, 8], [9, 9], [10, 10]])
# Apply KMeans with k = 2
kmeans = KMeans(n_clusters=2, random_state=42)
kmeans.fit(X)
# Get labels and cluster centers
labels = kmeans.labels_
centers = kmeans.cluster_centers_
# Visualization
plt.figure(figsize=(6, 5))
plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='coolwarm', s=100, label='Points')
plt.scatter(centers[:, 0], centers[:, 1], c='black', s=200, marker='X', label='Centroids'
plt.title("K-Means Clustering with k=2")
plt.xlabel("X")
plt.ylabel("Y")
plt.legend()
plt.grid(True)
plt.show()
```

Output:



9. Write a Python script to perform K-Means clustering on the following dataset: Mall Customer Segmentation. Use $k = 5$ (also, determine optimal k via the Elbow Method) and visualize the clusters to identify customer segments.

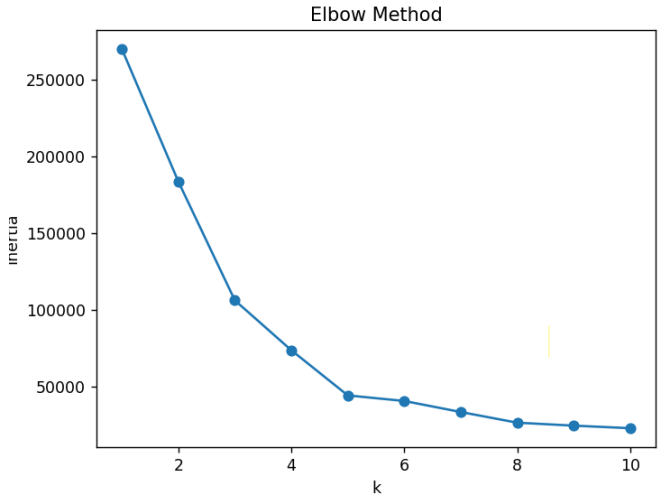
   Expected Output:

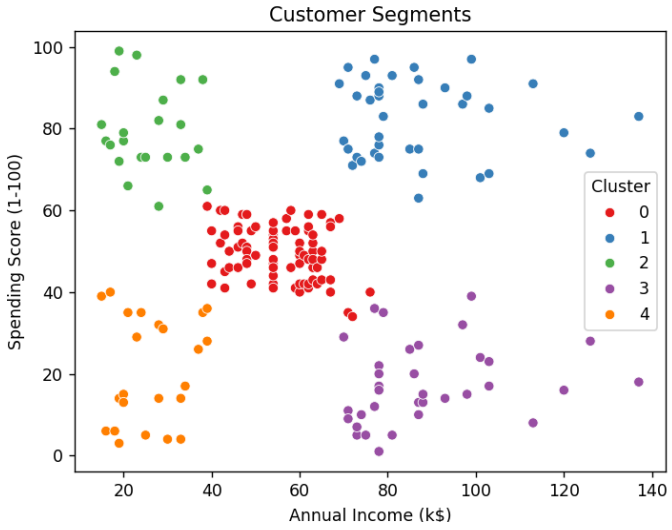
   - Scatter plot showing clusters (e.g., "High Income-Low Spenders," "Moderate Income-Moderate Spenders").
   - Insights for targeted marketing strategies.

Ans-

```
#Name -Arman Singh Regd=2241019468
import pandas as pd
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt
import seaborn as sns
df = pd.read_csv("D:/BIter 3rd Year 6th Sem/Python/MyAssignment/Ass4/Mall_Customers.csv")
X = df[['Annual Income (k$)', 'Spending Score (1-100)']]
inertia = []
for k in range(1, 11):
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(X)
    inertia.append(kmeans.inertia_)
plt.plot(range(1, 11), inertia, marker='o')
plt.xlabel('k')
plt.ylabel('Inertia')
plt.title('Elbow Method')
plt.show()

kmeans = KMeans(n_clusters=5, random_state=42)
df['Cluster'] = kmeans.fit_predict(X)

sns.scatterplot(x='Annual Income (k$)', y='Spending Score (1-100)', hue='Cluster', data=c
plt.title("Customer Segments")
plt.show()
```

Output





10. Perform the following tasks using the pandas `Series` object:

    (a) Create a Series from the list `[7, 11, 13, 17]`.

    (b) Create a Series with five elements where each element is `100.0`.

    (c) Create a Series with 20 elements that are all random numbers in the range 0 to 100. Use the `describe` method to produce the Series' basic descriptive statistics.

    (d) Create a Series called `temperatures` with the following floating-point values: `98.6`, `98.9`, `100.2`, and `97.9`. Use the `index` keyword argument to specify the custom indices 'Julie', 'Charlie', 'Sam', and 'Andrea'.

    (e) Form a dictionary from the names and values in Part (d), then use it to initialize a Series.

```python
#Name -Arman Singh Regd=2241019468
import pandas as pd
import numpy as np

# (a)
s1 = pd.Series([7, 11, 13, 17])
print(s1)

# (b)
s2 = pd.Series([100.0]*5)
print(s2)

# (c)
s3 = pd.Series(np.random.randint(0, 101, 20))
print(s3.describe())

# (d)
temperatures = pd.Series([98.6, 98.9, 100.2, 97.9], index=['Julie', 'Charlie', 'Sam', 'Ar
print(temperatures)

# (e)
temp_dict = {'Julie': 98.6, 'Charlie': 98.9, 'Sam': 100.2, 'Andrea': 97.9}
s_temp = pd.Series(temp_dict)
print(s_temp)
```

```
Iter 3rd Year 6th Sem\Python\MyAssignment\Ass4\q10.py"
0     7
1    11
2    13
3    17
dtype: int64
0    100.0
1    100.0
2    100.0
3    100.0
4    100.0
dtype: float64
count     20.000000
mean      58.000000
std       30.668001
min        3.000000
25%       34.000000
50%       56.500000
75%       86.000000
max      100.000000
dtype: float64
Julie       98.6
Charlie     98.9
Sam        100.2
Andrea      97.9
dtype: float64
Julie       98.6
Charlie     98.9
Sam        100.2
Andrea      97.9
dtype: float64
PS D:\BIter 3rd Year 6th Sem\Python\MyAssignment>
```